

Host-Based Intrusion Detection System Using the ELK Stack and Suricata

CONTENTS

INTRODUCTION	1
METHODOLOGY	3
INSTALLATION	4
CONFIGURATION	6
ELASTICSEARCH.....	6
KIBANA	8
LOGSTASH	13
SURICATA	16
FILEBEAT	18
DASHBOARD SETUP	22
CONCLUSION	24

INTRODUCTION

In this project, a Host-Based Intrusion Detection System (HIDS) was built by configuring Suricata together with the ELK Stack (Elasticsearch, Logstash, and Kibana) to capture, process, and visualize real-time network events on a Kali Linux host.

1) What is a HIDS?

A Host-Based Intrusion Detection System is a cybersecurity tool that monitors and analyses traffic to and from a host for suspicious activity, policy violations, or potential attacks in real time. When malicious activity is detected, the HIDS generates alerts for a security analyst to investigate.

2) What is the ELK Stack?

The ELK Stack is an open-source log management and data analysis platform used for collecting, storing, analysing, and visualizing logs from multiple sources. ELK stands for:

- **Elasticsearch:** A search and analytics engine that stores and indexes data for fast searching.
- **Logstash:** A data processing pipeline that collects, filters, and forwards data to Elasticsearch.
- **Kibana:** A visualization tool that displays data stored in Elasticsearch using dashboards and graphs.

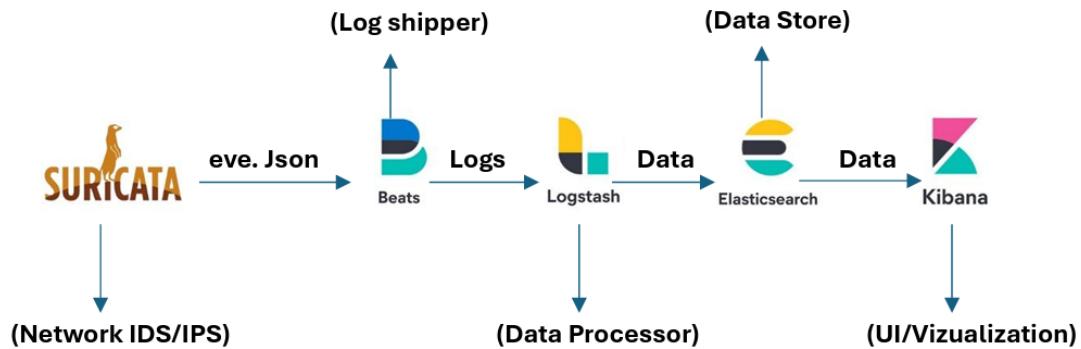
3) What is Suricata?

Suricata is an open-source Intrusion Detection System (IDS), Intrusion Prevention System (IPS), and Network Security Monitoring (NSM) engine developed by the Open Information Security Foundation (OISF).

It monitors network traffic in real time and inspects packets using rule-based signatures, logging any matches for further analysis.

Suricata generates the **eve.json** file through its **EVE (Extensible Event Format) logging module**, which is designed to output structured, machine-readable logs for use in analysis platforms like the ELK Stack.

Data flow in this project:



- Suricata writes EVE JSON logs to disk.
- Filebeat monitors these logs and sends them to Logstash.
- Logstash processes and enriches the logs.
- Elasticsearch stores and indexes the processed logs.
- Kibana queries Elasticsearch to visualize and analyse the Suricata event data.

Methodology

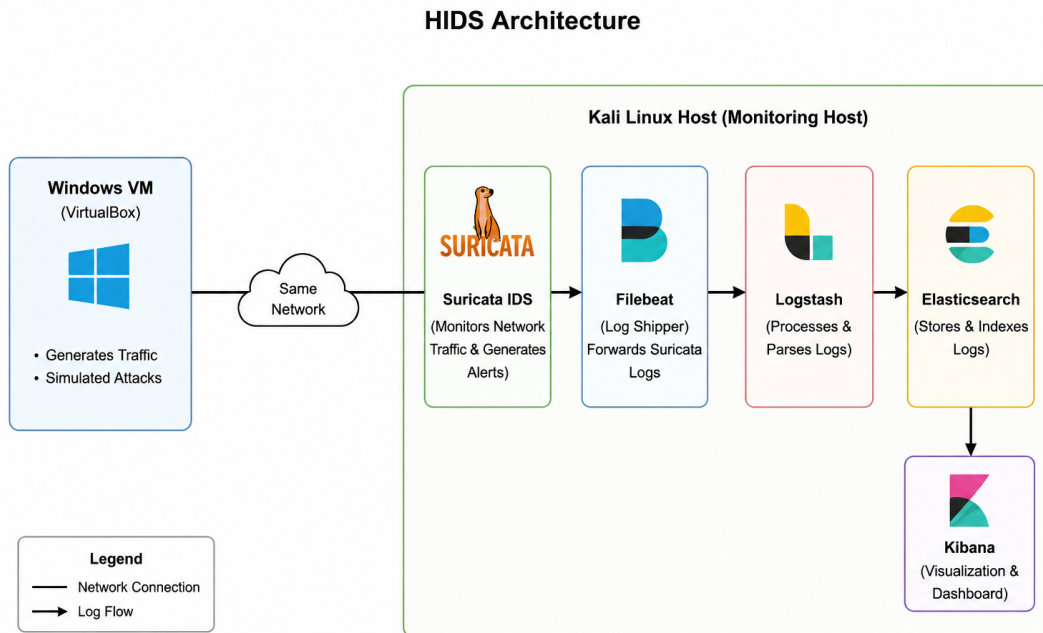
1) Tools Required:

- Elasticsearch
- Logstash
- Kibana
- Filebeat
- Suricata

2) OS Used:

This project uses **Kali Linux**, a Debian-based operating system, installed directly on the physical machine (bare metal) not inside a virtual machine. Kali Linux runs the complete monitoring and detection stack together.

A separate **Windows virtual machine**, running under VirtualBox, was used as a second device on the same local network. This Windows VM was used to generate test traffic pings, Nmap scans, and SSH connection attempts directed at the Kali host, allowing the HIDS rules to be tested against real traffic from another machine.



Both the monitoring stack (Kibana, Elasticsearch, Logstash) and the detection components (Filebeat, Suricata) run on the same Kali Linux machine. The Windows VM communicates with Kali over the local network (192.168.0.0/24), with Kali's wireless interface used by Suricata to capture all traffic.

Installation

1) Elasticsearch

Download Elasticsearch package from [elastic.co](https://www.elastic.co/downloads/elasticsearch)

<https://www.elastic.co/downloads/elasticsearch>

Here I am using kali linux so I am going with deb x86_64 package.

Install Elasticsearch:

```
dpkg -i package_name.deb
```

```
for eg. dpkg -i elasticsearch-9.1.5-amd64.deb
```

Start the Elasticsearch service:

```
sudo systemctl enable elasticsearch  
sudo systemctl start elasticsearch
```

To check the elasticsearch service status:

```
sudo systemctl status elasticsearch
```

2) Logstash

Download Logstash.

<https://www.elastic.co/downloads/logstash>

Install logstash:

```
dpkg -i package_name.deb
```

Start Logstash service:

```
sudo systemctl enable logstash  
sudo systemctl start logstash  
sudo systemctl status logstash
```

3) Kibana

Download Kibana package:

<https://www.elastic.co/downloads/kibana>

Install Kibana:

```
dpkg -i package_name.deb
```

Start Kibana service:

```
sudo systemctl enable kibana  
sudo systemctl start kibana  
sudo systemctl status kibana
```

Next download and install Filebeat and Suricata in host system.

4) Filebeat Download Filebeat package:

<https://www.elastic.co/downloads/beats/filebeat>

Install Filebeat:

```
dpkg -i package_name.deb
```

Start Filebeat service:

```
sudo systemctl enable filebeat  
sudo systemctl start filebeat  
sudo systemctl status filebeat
```

5) Suricata :

Install Suricata:

```
sudo apt install suricata
```

Start and Enable Suricata

```
sudo systemctl enable suricata  
sudo systemctl start suricata  
sudo systemctl status suricata
```

Next step is to configure these tools together by editing their yml/yaml, conf, rules file.

Note:

Don't forget to restart the service after editing their file if the services are Running

For eg. if you edited the elasticsearch.yml file then restart the service by

`sudo systemctl restart elasticsearch`

Configuration

1) Configure Elasticsearch

Go to `/etc/elasticsearch` directory and edit the `elasticsearch.yml`

```
sudo nano elasticsearch.yml
```

Uncomment required lines

Network.host: "0.0.0.0" #using 0.0.0.0 exposes the service to all network Connections

http.port: 9200

transport.host: "0.0.0.0"

```
# ----- Paths -----
# Path to directory where to store the data (separate multiple locations by comma):
path.data: /var/lib/elasticsearch
# Path to log files:
path.logs: /var/log/elasticsearch
# ----- Memory -----
# Lock the memory on startup:
bootstrap.memory_lock: true
# Make sure that the heap size is set to about half the memory available
# on the system and that the owner of the process is allowed to use this
# limit.
# Elasticsearch performs poorly when the system is swapping the memory.
# ----- Network -----
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
network.host: "0.0.0.0"
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
http.port: 9200
# For more information, consult the network module documentation.
# ----- Discovery -----
```

```
# Enable security features
xpack.security.enabled: true
xpack.security.enrollment.enabled: true
# Enable encryption for HTTP API client connections, such as Kibana, Logstash, and Agents
xpack.security.http.ssl:
  enabled: true
  keystore.path: certs/http.p12
# Enable encryption and mutual authentication between cluster nodes
xpack.security.transport.ssl:
  enabled: true
  verification_mode: certificate
  keystore.path: certs/transport.p12
  truststore.path: certs/transport.p12
# Create a new cluster with the current node only
# Additional nodes can still join the cluster later
cluster.initial_master_nodes: ["RA-7"]
# Allow HTTP API connections from anywhere
# Connections are encrypted and require user authentication
http.host: "0.0.0.0"
# Allow other nodes to join the cluster from anywhere
# Connections are encrypted and mutually authenticated
transport.host: "0.0.0.0"
#----- END SECURITY AUTO CONFIGURATION -----
```

Save and exit:

Ctrl+x

y, and enter

Enable and start the elasticsearch service, if already running then restart it.

```
sudo systemctl restart elasticsearch
```

If you want to reset the elastic password, then run this command

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
```

If you get a permission error then run these commands

```
sudo chown -R elasticsearch:elasticsearch /etc/elasticsearch
sudo chown -R 750 /etc/elasticsearch
```

Then retry:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
```

```
(senha@BLACKICE)-[~/Downloads]
$ sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
This tool will reset the password of the [elastic] user to an autogenerated value.
The password will be printed in the console.
Please confirm that you would like to continue [y/N]y

Password for the [elastic] user successfully reset.
New value: IJ=UzftfUc8IXNlE_Z0Q
```

Note it somewhere, don't forget the password.

Check the elasticsearch is working or not:

```
(senha@BLACKICE)-[~/Downloads]
$ curl -k -u elastic:IJ=UzftfUc8IXNlE_Z0Q https://localhost:9200
{
  "name" : "BLACKICE",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "RRUepGEsSdyoix5o5XoP8g",
  "version" : {
    "number" : "9.4.2",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "c402c2b36d90eae29c0182f86bd9050fd0b746cc",
    "build_date" : "2026-05-25T22:10:36.017759931Z",
    "build_snapshot" : false,
    "lucene_version" : "10.4.0",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

Your Elasticsearch instance is running perfectly if you got a result like this.

2) Configure Kibana

Go to **/etc/kibana** directory and run this command

```
sudo nano kibana.yml
```

Uncomment these lines in **System: Kibana server** section

```
server.port: 5061
```

```
server.host: "0.0.0.0"
```

```
GNU nano 8.6 /etc/kibana/kibana.yml *
# For more configuration options see the configuration guide for Kibana in
# https://www.elastic.co/guide/index.html

# ----- System: Kibana Server -----
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5061

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the 'server.rewriteBasePath' setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

# Specifies whether Kibana should rewrite requests that are prefixed with
# 'server.basePath' or require that they are rewritten by your reverse proxy.
# Defaults to 'false'.
#server.rewriteBasePath: false

# Specifies the public URL at which Kibana is available for end users. If
# 'server.basePath' is configured this URL should end with the same basePath.
#server.publicBaseUrl: ""

# The maximum payload size in bytes for incoming server requests.
#server.maxPayload: 1048576

# The Kibana server's name. This is used for display purposes.
server.name: "kibana"

# ----- System: Kibana Server (Optional) -----
# Enables SSL and paths to the PEM-format SSL certificate and SSL key files, respectively.

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark M-J To Bracket
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^_ Justify   ^/ Go To Line M-E Redo     M-B Copy     ^B Where Was
```

In **System: Elasticsearch** Section, give **elasticsearch.hosts**, elasticsearch's username and password

```
# ===== System: Elasticsearch =====
# The URLs of the Elasticsearch instances to use for all your queries.
elasticsearch.hosts: ["https://localhost:9200"]

# If your Elasticsearch is protected with basic authentication, these settings provide
# the username and password that the Kibana server uses to perform maintenance on the Kibana
# index at startup. Your Kibana users still need to authenticate with Elasticsearch, which
# is proxied through the Kibana server.
elasticsearch.username: "elastic"
elasticsearch.password: "IJ=UzftfUc8IXNLE_Z0Q"

# Kibana can also authenticate to Elasticsearch via "service account tokens".
# Service account tokens are Bearer style tokens that replace the traditional username/password based configuration.
# Use this token instead of a username/password.
# elasticsearch.serviceAccountToken: "my_token"

# Time in milliseconds to wait for Elasticsearch to respond to pings. Defaults to the value of
# the elasticsearch.requestTimeout setting.
#elasticsearch.pingTimeout: 1500

# Time in milliseconds to wait for responses from the back end or Elasticsearch. This value
# must be a positive integer.
#elasticsearch.requestTimeout: 30000

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M] To Bracket
^X Exit      ^R Read File  ^_ Replace    ^V Paste      ^J Justify    ^_/ Go To Line M-E Redo      M-G Copy      ^B Where Was
```

Or use **Elasticsearch.ServiceAccountToken** instead of username and password.

Generate **Elasticsearch.ServiceAccountToken** using this command

`sudo /usr/share/elasticsearch/bin/elasticsearch-service-tokens create elastic/kibana kibana-token`

```
# ===== System: Elastisearch =====
# The URLs of the Elastisearch instances to use for all your queries.
elastisearch.hosts: ["https://localhost:9200"]

# If your Elastisearch is protected with basic authentication, these settings provide
# the username and password that the Kibana server uses to perform maintenance on the Kibana
# index at startup. Your Kibana users still need to authenticate with Elastisearch, which
# is proxied through the Kibana server.
#elastisearch.username: "elastic"
elastisearch.password: "IJ=UzftfUc8IXNLE_Z0Q"

# Kibana can also authenticate to Elastisearch via "service account tokens".
# Service account tokens are Bearer style tokens that replace the traditional username/password based configuration.
# Use this token instead of a username/password.
elastisearch.serviceAccountToken: "AAEAAWVSyXN0aWmVa2liYw5hL2tpYmFuYS10b2tlbjoxNnB1NnpXOVFzZUx1b290eWlLaHB3"

# Time in milliseconds to wait for Elastisearch to respond to pings. Defaults to the value of
# the elasticsearch.requestTimeout setting.
#elastisearch.pingTimeout: 1500

# Time in milliseconds to wait for responses from the back end or Elastisearch. This value
```

Add Kibana's encryption keys

Run this command to generate encryption keys

```
sudo /usr/share/kibana/bin/kibana-encryption-keys generate
```

```
## Kibana Encryption Key Generation Utility

The 'generate' command guides you through the process of setting encryption keys for:

xpack.encryptedSavedObjects.encryptionKey
  Used to encrypt stored objects such as dashboards and visualizations
  https://www.elastic.co/guide/en/kibana/current/xpack-security-secure-saved-objects.html#xpack-security-secure-saved-objects

xpack.reporting.encryptionKey
  Used to encrypt saved reports
  https://www.elastic.co/guide/en/kibana/current/reporting-settings-kb.html#general-reporting-settings

xpack.security.encryptionKey
  Used to encrypt session information
  https://www.elastic.co/guide/en/kibana/current/security-settings-kb.html#security-session-and-cookie-settings

Already defined settings are ignored and can be regenerated using the --force flag. Check the documentation links for instructions on how to rotate encryption keys.
Definitions should be set in the kibana.yml used to configure Kibana.

Settings:
xpack.encryptedSavedObjects.encryptionKey: 7abf3c9d8e6a4f012b3d5e7f9a0c2d3e
xpack.reporting.encryptionKey: 8c1d4e6f0a2b5c7d9e3f6a8b0c1d4e5f
xpack.security.encryptionKey: 9d2e5f7a0b1c4d6e8f2a5b7c0d3e6f8a
```

In kibana.yml

```
GNU nano 8.6 /etc/kibana/kibana.yml *
# The path where Kibana stores persistent data not saved in Elasticsearch. Defaults to data
#path.data: data

# Specifies the path where Kibana creates the process ID file.
pid.file: /run/kibana/kibana.pid

# Set the interval in milliseconds to sample system and process performance
# metrics. Minimum is 100ms. Defaults to 5000ms.
#ops.interval: 5000

# Specifies locale to be used for all localizable strings, dates and number formats.
# Supported languages are the following: English (default) "en", Chinese "zh-CN", Japanese "ja-JP", French "fr-FR".
#i18n.locale: "en"

# --- Encryption Keys ---
xpack.encryptedSavedObjects.encryptionKey: 7abf3c9d8e6a4f012b3d5e7f9a0c2d3e
xpack.reporting.encryptionKey: 8c1d4e6f0a2b5c7d9e3f6a8b0c1d4e5f
xpack.security.encryptionKey: 9d2e5f7a0b1c4d6e8f2a5b7c0d3e6f8a

# ===== Frequently used (Optional) =====

# ===== Saved Objects: Migrations =====
# Saved object migrations run at startup. If you run into migration-related issues, you might need to adjust these settings.
```

Configure Elasticsearch CA Certificate in Kibana

Since Elasticsearch uses HTTPS, Kibana must verify Elasticsearch's identity using its CA certificate. Without this, Kibana cannot establish a secure connection to Elasticsearch.

Go to **/etc/elasticsearch/certs** directory. From this folder, copy the **http_ca.crt** file to Kibana's configuration folder and rename it as **ca.crt**:

```
sudo cp /etc/elasticsearch/certs/http_ca.crt /etc/kibana/ca.crt
```



```
(root@BLACKICE) - [/home/senha]
# ls
http_ca.crt http.p12 transport.p12
```

Add CA certificate path to kibana.yml:

```
sudo nano /etc/kibana/kibana.yml
```

```
elasticsearch.ssl.certificateAuthorities: ["/etc/kibana/ca.crt"]
```

```
elasticsearch.ssl.verificationMode: full
```

```
GNU nano 8.6 /etc/kibana/kibana.yml *
#elasticsearch.compression: false

# List of Kibana client-side headers to send to Elasticsearch. To send *no* client-side
# headers, set this value to [] (an empty list).
#elasticsearch.requestHeadersWhitelist: [ authorization ]

# Header names and values that are sent to Elasticsearch. Any custom headers cannot be overwritten
# by client-side headers, regardless of the elasticsearch.requestHeadersWhitelist configuration.
#elasticsearch.customHeaders: {}

# Time in milliseconds for Elasticsearch to wait for responses from shards. Set to 0 to disable.
#elasticsearch.shardTimeout: 30000

# ===== System: Elasticsearch (Optional) =====
# These files are used to verify the identity of Kibana to Elasticsearch and are required when
# xpack.security.http.ssl.client_authentication in Elasticsearch is set to required.
#elasticsearch.ssl.certificate: /path/to/your/client.crt
#elasticsearch.ssl.key: /path/to/your/client.key

# Enables you to specify a path to the PEM file for the certificate
# authority for your Elasticsearch instance.
elasticsearch.ssl.certificateAuthorities: [ "/etc/kibana/ca.crt" ]

# To disregard the validity of SSL certificates, change this setting's value to 'none'.
elasticsearch.ssl.verificationMode: full

# ===== System: Logging =====
# Set the value of this setting to off to suppress all logging output, or to debug to log everything. Defaults to 'info'
#logging.root.level: debug

# Enables you to specify a file where Kibana stores log output.
logging:
  appenders:
    file:
      type: file

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Set Mark  M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-F Redo     M-6 Copy      ^B Where Was
```

Save and exit.

Restart Kibana:

```
sudo systemctl restart kibana
```

Why use a CA certificate?

When Kibana connects to Elasticsearch over HTTPS, it must verify that the Elasticsearch server is legitimate and not an impostor or attacker pretending to be Elasticsearch. The CA certificate acts as a trusted proof of identity without it, Kibana would either reject the connection or connect insecurely.

Then open your browser and go to:

<http://localhost:5061>

Login with your elasticsearch's username and password.

The image shows two screenshots of a web browser. The top screenshot is the Elasticsearch login page at <http://localhost:5061/login?next=%2Fapp%2Fhome#/>. It features the Elastic logo and the text "Welcome to Elastic". There are input fields for "Username" (containing "elastic") and "Password" (masked with dots). A "Log In" button is at the bottom.

The bottom screenshot is the Elastic home dashboard at <http://localhost:5061/app/home#/>. It has a search bar at the top with the text "Find apps, content, and more." and a button "AI Agent". The main content area is titled "Welcome home" and contains four cards: "Elasticsearch" (Create search experiences with a refined set of APIs and tools), "Observability" (Consolidate your logs, metrics, application traces, and system availability with purpose-built UIs), "Security" (Prevent, collect, detect, and respond to threats for unified protection across your infrastructure), and "Analytics" (Explore, visualize, and analyze your data using a powerful suite of analytical tools and applications). Below these cards is a section "Get started by adding integrations" with a description and three buttons: "Add integrations", "Try sample data", and "Upload a file". To the right is a "Cloud Connect" section with a description and a "Get started" button. At the bottom is a "Management" section with four cards: "Manage permissions" (Control who has access and what tasks they can perform), "Monitor the stack" (Track the real-time health and performance of your), "Back up and restore" (Save snapshots to a backup repository, and restore to), and "Manage index lifecycles" (Define lifecycle policies to automatically perform).

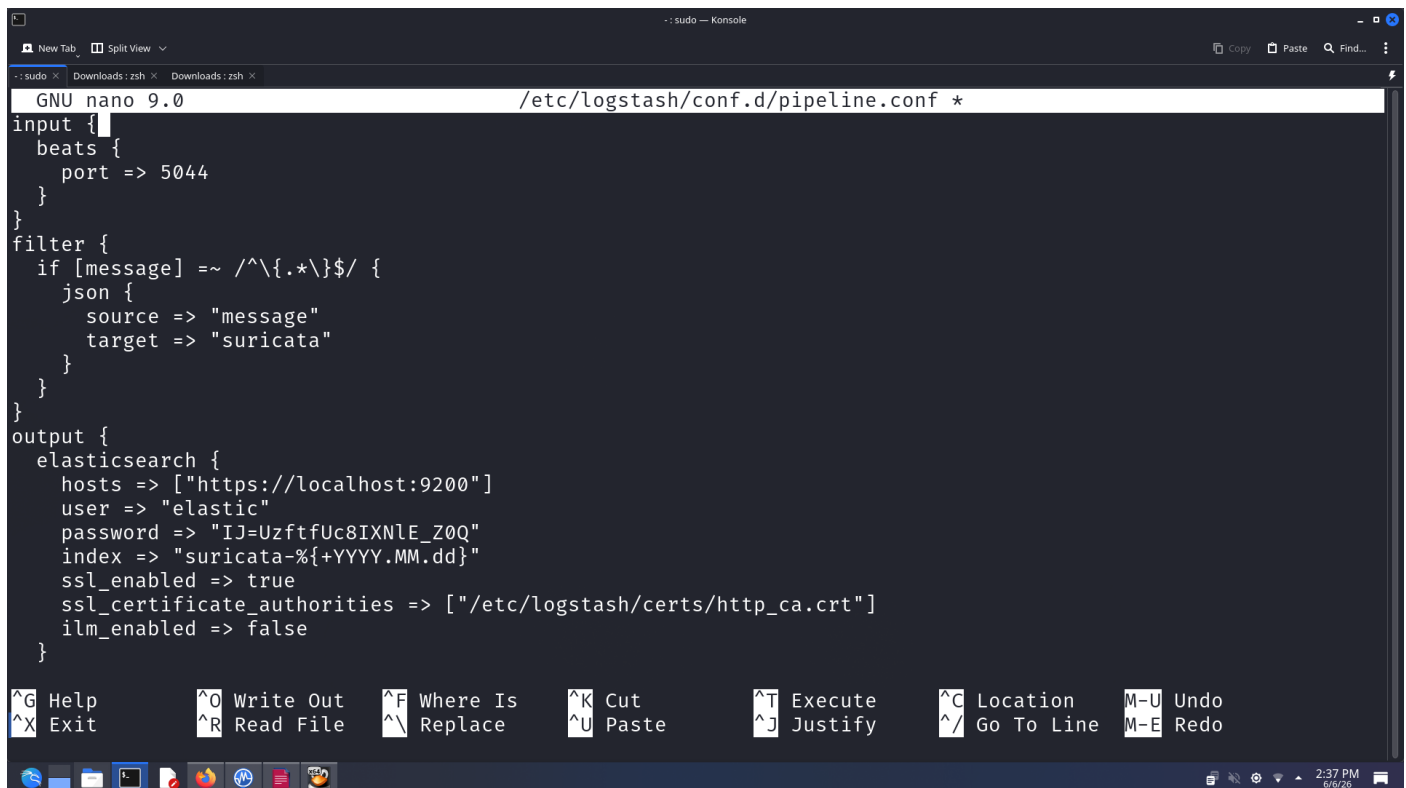
3) Configure Logstash

First make a pipeline configuration. This configuration file is the core instruction manual that tells Logstash which data to get (Input), how to clean it up (Filter), and where to send the final result (Output).

Go to **/etc/logstash/conf.d** directory and create the pipeline file:

```
sudo nano /etc/logstash/conf.d/pipeline.conf
```

Set the input and filter section like this:

A screenshot of a terminal window titled 'sudo - Konsole'. The terminal shows the nano text editor editing the file '/etc/logstash/conf.d/pipeline.conf'. The configuration is as follows:

```
input {
  beats {
    port => 5044
  }
}
filter {
  if [message] =~ /^{\.*}$/ {
    json {
      source => "message"
      target => "suricata"
    }
  }
}
output {
  elasticsearch {
    hosts => ["https://localhost:9200"]
    user => "elastic"
    password => "IJ=UzftfUc8IXNlE_Z0Q"
    index => "suricata-%{+YYYY.MM.dd}"
    ssl_enabled => true
    ssl_certificate_authorities => ["/etc/logstash/certs/http_ca.crt"]
    ilm_enabled => false
  }
}
```

The terminal window includes a status bar at the bottom with various keyboard shortcuts like ^G Help, ^X Exit, ^O Write Out, etc. The system tray at the bottom right shows the time as 2:37 PM on 6/6/26.

Host => ["hostname/ip_address:9200"]

Password => "your elasticsearch password"

ssl_certificate_authorities => get this from /etc/elasticsearch/certs/http_ca.crt

Copy the Elasticsearch CA certificate to Logstash:

```
sudo mkdir -p /etc/logstash/certs
```

```
sudo cp /etc/elasticsearch/certs/http_ca.crt /etc/logstash/certs/
```

```
sudo chown logstash:logstash /etc/logstash/certs/http_ca.crt
```



```
sudo chmod 644 /etc/logstash/certs/http_ca.crt
```

Fix Logstash directory permissions:

```
sudo chown -R logstash:logstash /var/lib/logstash
```

```
sudo chown -R logstash:logstash /var/log/logstash
```

Note: Logstash cannot be run as root. Always run it as a managed systemd service.

Enable and start Logstash:

```
sudo systemctl enable logstash
```

```
sudo systemctl start logstash
```

```
sudo systemctl status logstash
```

Confirm Logstash is listening on port 5044:

```
sudo ss -tulnp | grep 5044
```

```
(senha@BLACKICE)-[~/Downloads]
$ sudo ss -tulnp | grep 5044
tcp    LISTEN 0      4096      *:5044      *:*    users:(("java",pid=898,fd=186))
```

4) Configure Suricata

In this project the active interface was **wlan0** (IP address 192.168.0.105). Go to **/etc/suricata** and edit **suricata.yaml**:

```
sudo nano /etc/suricata/suricata.yaml
```

Set the interface in the af-packet section:

```
# Linux high speed capture support
af-packet:
- interface: wlan0
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
  # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.
  # This is only supported for Linux kernel > 3.1
  # possible value are:
  # * cluster_flow: all packets of a given flow are sent to the same socket
  # * cluster_cpu: all packets treated in kernel by a CPU are sent to the same socket
  # * cluster_qm: all packets linked by network card to a RSS queue are sent to the same
```

Confirm eve-log output is enabled:

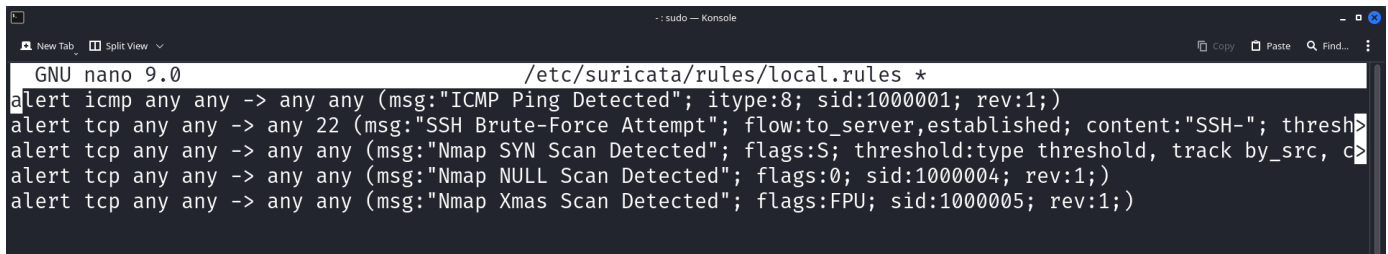
```
# Extensible Event Format (nicknamed EVE) event log in JSON format
- eve-log:
  enabled: yes
  filetype: regular #regular/syslog/unix_dgram/unix_stream/redis
  filename: /var/log/suricata/eve.json
  # Enable for multi-threaded eve.json output; output files are amended with
  # an identifier, e.g., eve.9.json
  #threaded: false
  # Specify the amount of buffering, in bytes, for
  # this output type. The default value 0 means "no
  # buffering".
  #buffer-size: 0
  #prefix: "@cee: " # prefix to prepend to each log entry
```

Create the rules file

Go to **/etc/suricata/rules** directory and create **local.rules**:

```
sudo nano /etc/suricata/rules/local.rules
```

Add these five rules:



```
GNU nano 9.0 /etc/suricata/rules/local.rules *
alert icmp any any -> any any (msg:"ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)
alert tcp any any -> any 22 (msg:"SSH Brute-Force Attempt"; flow:to_server,established; content:"SSH-"; thresh>
alert tcp any any -> any any (msg:"Nmap SYN Scan Detected"; flags:S; threshold:type threshold, track by_src, c>
alert tcp any any -> any any (msg:"Nmap NULL Scan Detected"; flags:0; sid:1000004; rev:1;)
alert tcp any any -> any any (msg:"Nmap Xmas Scan Detected"; flags:FPU; sid:1000005; rev:1;)
```

Note: Every rule's sid must be unique, otherwise it won't work.

Set the rules file in **suricata.yaml**:

```
##
## Configure Suricata to load Suricata-Update managed rules.
##

default-rule-path: /var/lib/suricata/rules

rule-files:
- local.rules

##
## Auxiliary configuration files.
##

classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config
# threshold-file: /etc/suricata/threshold.config

##
```

Copy rules to the correct directory (Suricata's default-rule-path is `/var/lib/suricata/rules/`):

```
sudo cp /etc/suricata/rules/local.rules /var/lib/suricata/rules/
```

Run this command to check the rules are working properly:

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```

```
(senha@BLACKICE)~[~/Downloads]
$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 8.0.5 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 12
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: /var/log/suricata/eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 1 rule files processed. 5 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 5 signatures processed. 0 are IP-only rules, 1 are inspecting packet payload, 0 inspect applicati
on layer, 0 are decoder event only
Notice: suricata: Configuration provided was successfully loaded. Exiting.
```

Check the eve.json is being generated:

```
(senha@BLACKICE)~[~/Downloads]
$ sudo tail -f /var/log/suricata/eve.json
{"timestamp":"2026-06-08T14:39:25.684744+0530","flow_id":1533580977827742,"in_iface":"wlan0","event_type":"quic
","src_ip":"192.168.0.105","src_port":60041,"dest_ip":"208.103.161.2","dest_port":443,"proto":"UDP","ip_v":4,"p
kt_src":"wire/pcap","quic":{"version":"1"}}
{"timestamp":"2026-06-08T14:39:25.684798+0530","flow_id":1533580977827742,"in_iface":"wlan0","event_type":"quic
","src_ip":"192.168.0.105","src_port":60041,"dest_ip":"208.103.161.2","dest_port":443,"proto":"UDP","ip_v":4,"p
kt_src":"wire/pcap","quic":{"version":"1","sni":"app.notion.com","ja3":{"hash":"81f9884a0c0c0d940d360491988c864
b","string":"771,4865-4867-4866,16-0-43-45-51-28-10-34-13-23-65281-57-65037,4588-29-23-24-25","ja4":"q13d03
14h3_55b375c5d22e_61e396c58b1f","extensions":[{"name":"alpn","type":16,"values":["h3"]},{"name":"server_name","
type":0,"values":["app.notion.com"]},{"name":"supported_versions","type":43,"values":["Tls13"]},{"name":"psk_ke
y_exchange_modes","type":45},{"name":"status_request","type":5},{"name":"key_share","type":51},{"name":"record_
size_limit","type":28},{"name":"supported_groups","type":10},{"name":"delegated_credentials","type":34},{"name"
:"signature_algorithms","type":13,"values":["1027","1283","1539","515","2052","2053","2054","1025","1281","1537
","513"]},{"name":"extended_master_secret","type":23},{"name":"renegotiation_info","type":65281},{"name":"quic_
transport_parameters","type":57},{"type":65037}]}}
```

5) Configure Filebeat

Go to `/etc/filebeat` and edit `filebeat.yml`:

```
sudo nano /etc/filebeat/filebeat.yml
```

Go to the `filebeat.inputs` section and set the paths.

Note: Filebeat 9 does not support the old `log` input type use `filestream` instead, otherwise the service will fail to start:

```

filebeat.inputs:

# Each - is an input. Most options can be set at the input level, so
# you can use different inputs for various configurations.
# Below are the input-specific configurations.

# filestream is an input for collecting log messages from files.
- type: filestream

  # Unique ID among all inputs, an ID is required.
  id: my-filestream-id

  # Change to true to enable this input configuration.
  enabled: true

  # Paths that should be crawled and fetched. Glob based paths.
  paths:
    - /var/log/suricata/eve.json
    - /var/log/auth.log
    #- c:\programdata\elasticsearch\logs\*

```

Next comment out the Elasticsearch output section and uncomment the Logstash output section.
 Note: Filebeat will not start if more than one output is active at the same time:

```

# ----- Elasticsearch Output -----
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["localhost:9200"]

  # Performance preset - one of "balanced", "throughput", "scale",
  # "latency", or "custom".
  preset: balanced

  # Protocol - either `http` (default) or `https`.
  #protocol: "https"

  # Authentication credentials - either API key or username/password.
  #api_key: "id:api_key"
  #username: "elastic"
  #password: "changeme"

```

Fix file permissions so Filebeat can read the log files:

```

sudo chmod 644 /var/log/suricata/eve.json
sudo chmod 644 /var/log/auth.log

```

Check the connection between Filebeat and Logstash. Logstash must be running otherwise it won't connect:

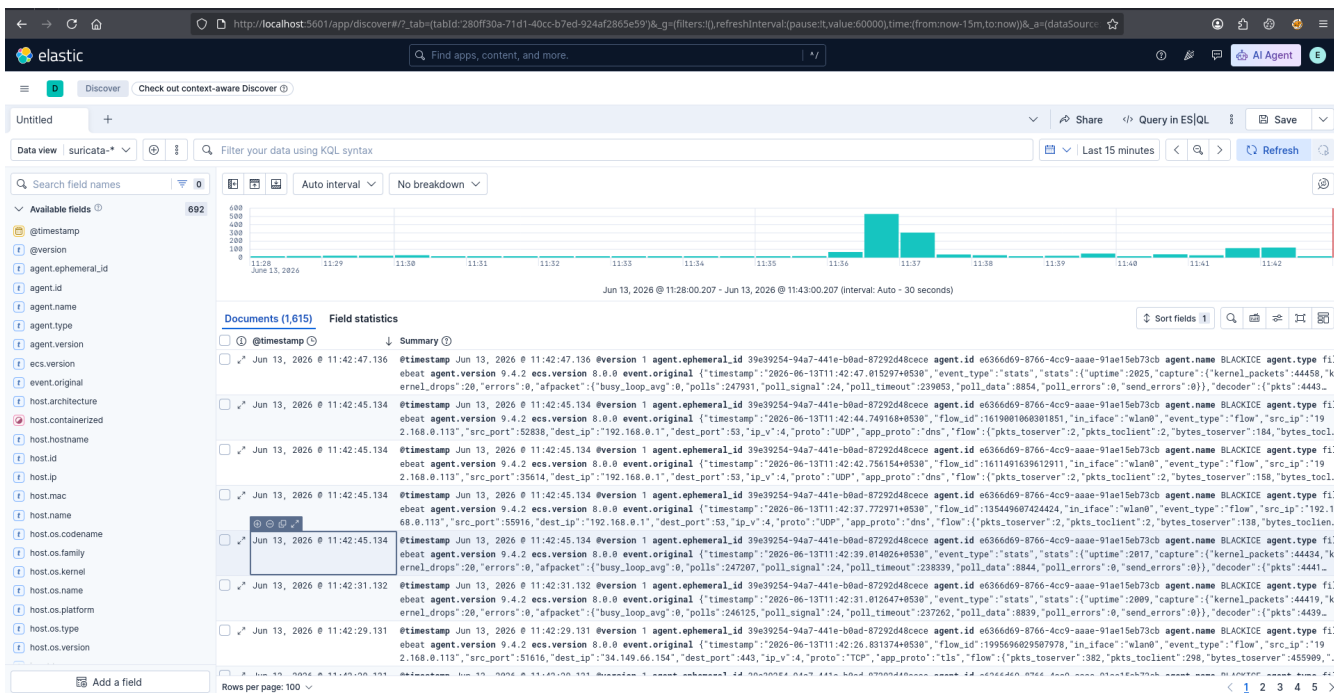
```
(senha@BLACKICE)-[~/Downloads]
$ sudo filebeat test output
logstash: localhost:5044...
connection...
parse host... OK
dns lookup... OK
addresses: ::1, 127.0.0.1
dial up... OK
TLS... WARN secure connection disabled
talk to server... OK
```

Check for Logs in Kibana

Go to Kibana → Stack Management → Data Views → Create data view:

- Index pattern: `suricata-*`
- Timestamp field: `@timestamp`
- Save the data view

Then go to Analytics → Discover. Select the `suricata-*` data view. You should see over 1,600 documents appearing.



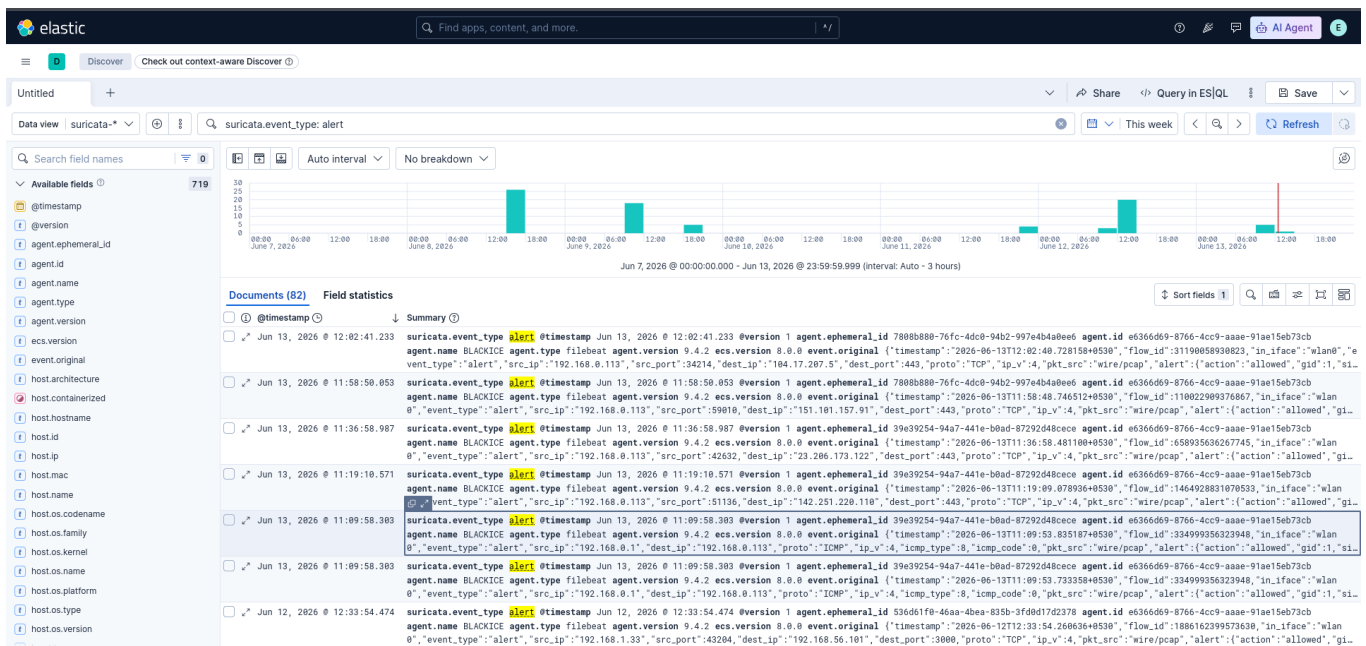
Now check the Suricata rules are triggered. From your Windows VM, ping the Kali host:

```
ping 192.168.0.105 -n 20
```

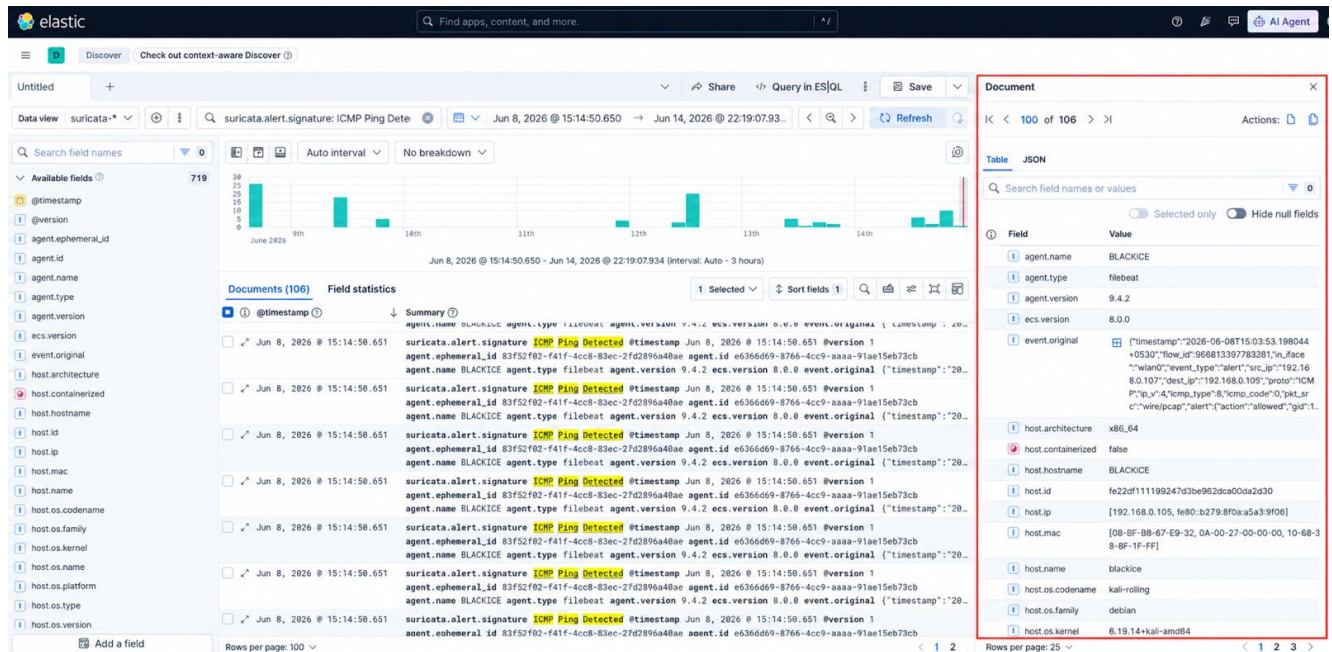
On Kali, check fast.log to confirm the alert fired:

```
(senha@BLACKICE)-[~/Downloads]
$ sudo cat /var/log/suricata/fast.log
06/08/2026-15:03:47.094031  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
06/08/2026-15:03:48.109252  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
06/08/2026-15:03:49.120170  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
06/08/2026-15:03:50.137587  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
06/08/2026-15:03:51.142700  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
06/08/2026-15:03:52.169010  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
06/08/2026-15:03:53.198044  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
06/08/2026-15:03:54.210937  [**] [1:1000001:1] ICMP Ping Detected [**] [Classification: (null)] [Priority: 3] {
ICMP} 192.168.0.107:8 -> 192.168.0.105:0
```

Now filter for alert events in Kibana Discover:



Expanded alert document:



You can set alerts in the free Kibana version, but your options are limited to the features available with the basic license. With the free version, you can create the rules themselves like index threshold or query rules, but you are restricted to using only a limited set of actions for notifications, primarily logging to an index. Sending notifications through more advanced channels like Slack, email, or webhooks requires a paid commercial license.

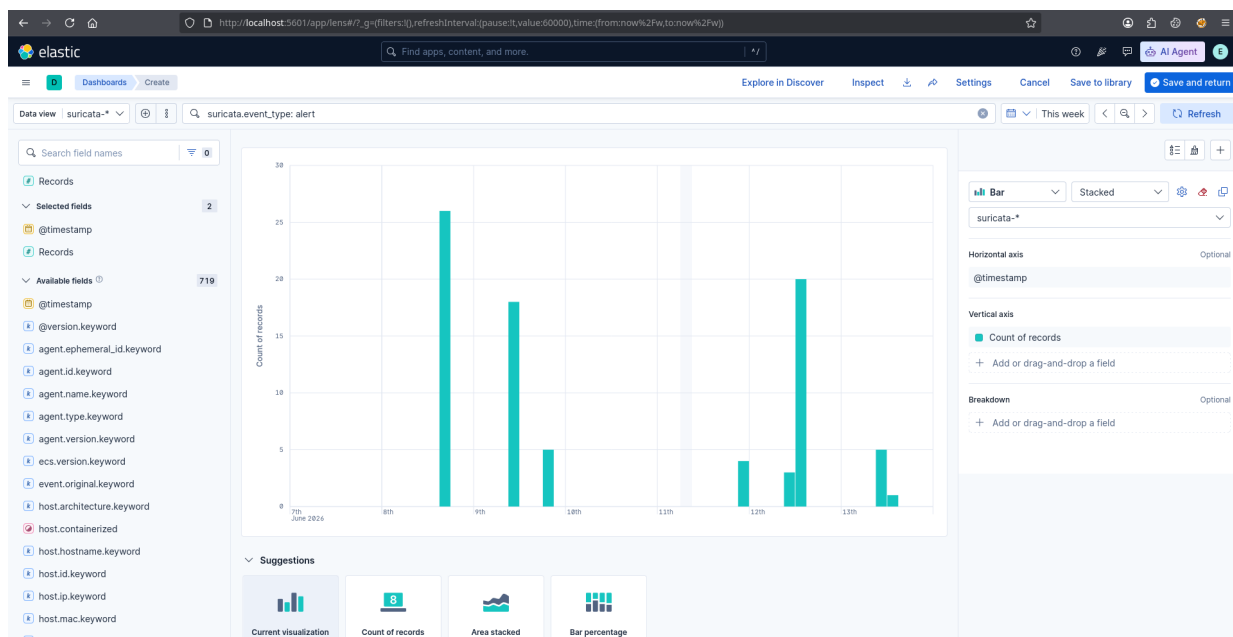
Dashboard Setup

1) Create Dashboard

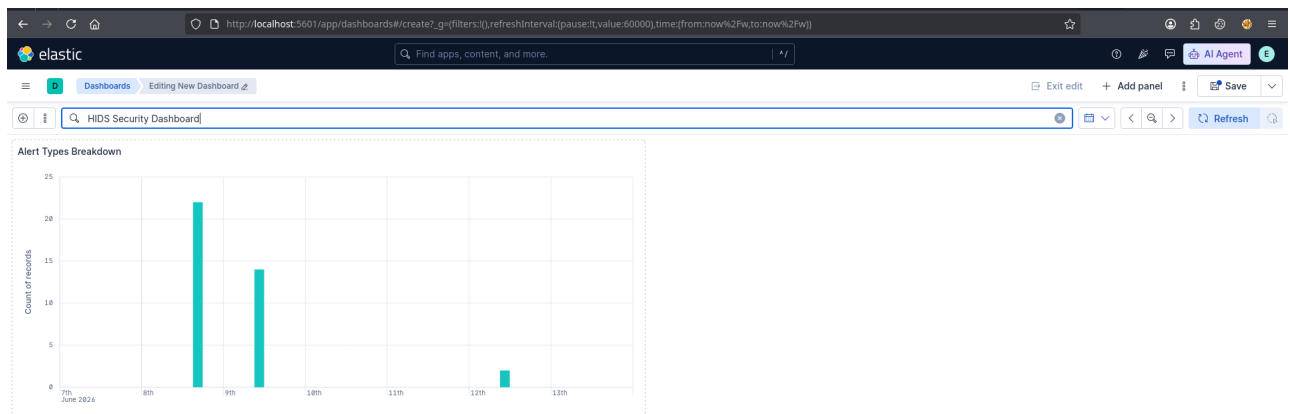
- Kibana → Analytics → Dashboards
- Click **Create dashboard**
- Click **Save**, then give a name for your dashboard and save it.

2) Create Visualization

- Analytics → Visualize Library → Create visualization → **Lens**
- Choose your data view: **suricata-***
- In the filter bar at the top type: **suricata.event_type: alert**
- Set the visualization type to **Bar**
- Drag **@timestamp** to the **Horizontal axis**
- Drag **Records** to the **Vertical axis** (it will show as Count of records)
- Click **Save and return**
- Give it the name **Alert Types Breakdown**
- Add to your dashboard and save



The chart clearly displays spikes of alert activity corresponding to the testing periods. The highest spike showed over 20 alerts in a single time bucket on June 8th when the Windows VM was actively pinging the Kali host.



Conclusion

In this project, a Host-Based Intrusion Detection System (HIDS) was successfully built and tested using Suricata and the ELK Stack (Elasticsearch, Logstash, and Kibana) on a Kali Linux machine. Suricata was installed and configured to monitor live network traffic on the wireless interface (wlan0). Five custom detection rules were written in local.rules to detect ICMP ping attempts, SSH brute-force attacks, and Nmap scan techniques. The rules were validated using Suricata's test mode which confirmed all 5 rules were successfully loaded.

A Windows virtual machine running inside VirtualBox was used as the attack source. When the Windows VM (192.168.0.107) pinged the Kali host (192.168.0.105), Suricata immediately detected and logged the alerts in fast.log.

Filebeat was configured to ship the Suricata eve.json logs to Logstash on port 5044. Logstash parsed and forwarded the events to Elasticsearch. The pipeline was fully verified documents were indexed in Elasticsearch and 82 alert events were confirmed in Kibana Discover using the filter `suricata.event_type: alert`.

A Kibana dashboard was created with a bar chart visualization titled "Alert Types Breakdown" showing alert activity over time. The highest spike showed over 20 alerts on June 8th during active testing.

Several real challenges were encountered and solved during this project, including Kibana rejecting the elastic superuser account, Elasticsearch file permission errors, Logstash permission issues, Suricata rule path problems, and Filebeat deprecated input type errors. Each challenge was resolved through careful troubleshooting and provided valuable real-world experience.

While Suricata is primarily designed as a Network Intrusion Detection and Prevention System (NIDS/NIPS), this project shows that when deployed on a host's own network interface it can effectively monitor host-level traffic. In a production environment, deploying Suricata on a network gateway or SPAN port would extend this same approach to an entire network. This project successfully demonstrated how open-source tools can be integrated together to build a working real-time intrusion detection and visualization system from scratch.